



Basics of Distributed Version Control

Introduction to DevOps

Joonas Rynänen

LUT University

 Introduction to DevOps

CONTENTS

What is Version Control?

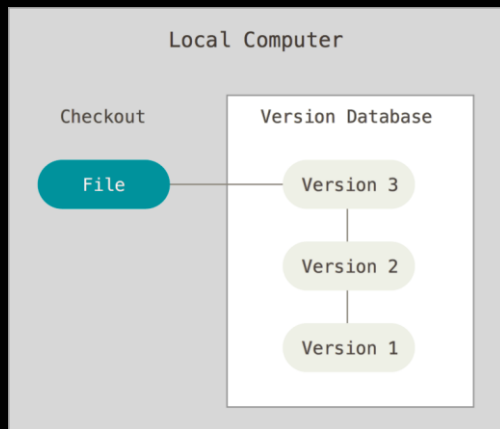
- Version Control is a technique to keep track of changes in a file or a set of files
- Version Control system records how the file(s) have changed, and keeps a store of the different versions
- Version Control is useful when developing software because it allows to easily switch between the versions
 - For example, if the latest version of the software introduces a bug that takes considerable amount of time to fix, it is convenient to revert back to the old version that works

Types of Version Control Systems

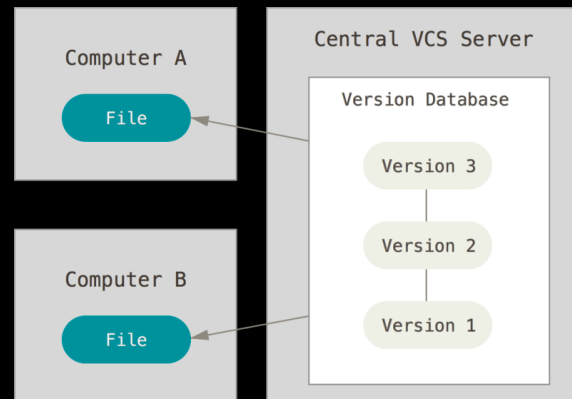
- Local version control
 - Includes a local version database that contains data about the file changes of each version
 - Eg. RCS (<https://www.gnu.org/software/rcs/>)
- Centralized version control
 - Includes a central server holding the version information
 - Allows for collaboration (server can be accessed by all the developers)
 - Better than local version control, but it still has some downsides (eg. single point of failure)
 - Eg. Subversion (<https://subversion.apache.org/>)
- Distributed version control
 - Each developer holds a mirror of the whole version history
 - If the VC server dies, it is possible to recreate by the clients
 - Local version history should be “synced” periodically with the remote server in order to keep other developers up-to-date

Types of Version Control Systems

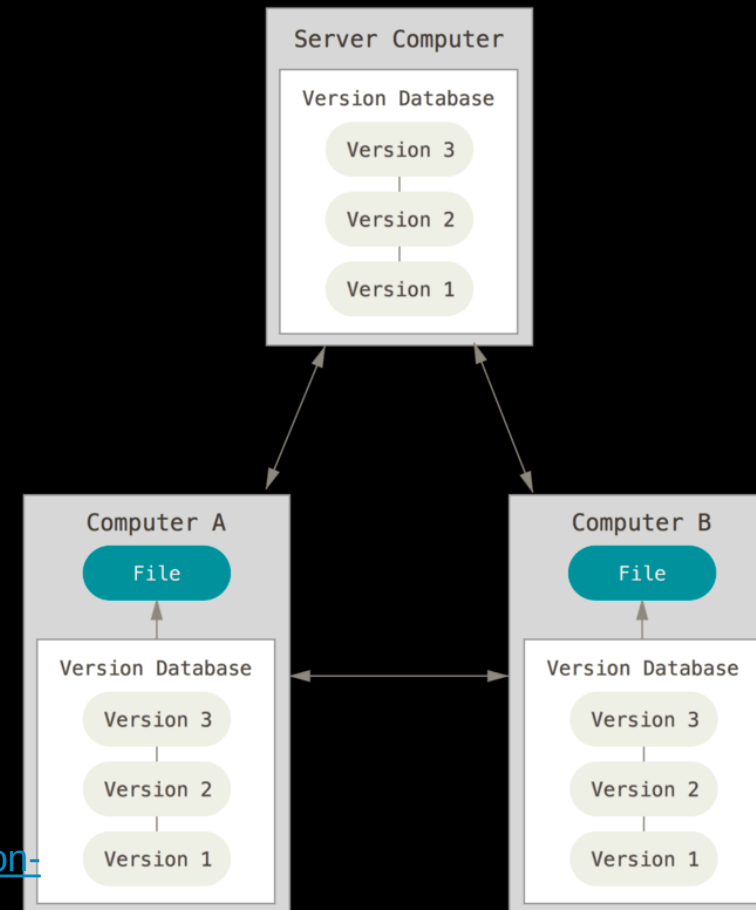
Distributed version control



Local version control



Centralized version control



Basics of Git

- Snapshots, not differences: <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>
 - Git basically takes a picture from all the files (a snapshot) and saves a reference to it
- Nearly every operation is local
- Integrity: it's impossible to change the contents of a file without Git recognizing it
- Git generally only adds data, meaning that most of the operation can be undone
- Files have three different states:
 - Modified: the file is changed but not yet committed to the version database
 - Staged: the file is modified and marked to be stored to the database on the next commit (no more modifications to be done to the file)
 - Committed: the file is stored in the local version database
- The whole project has three different sections:
 - Working tree: a single checkout of a certain version of the project
 - Staging area: a file containing information about the next commit
 - Git directory: the place where Git stores the metadata and the object database of the project

Basic Git Workflow

Modify files in the working tree



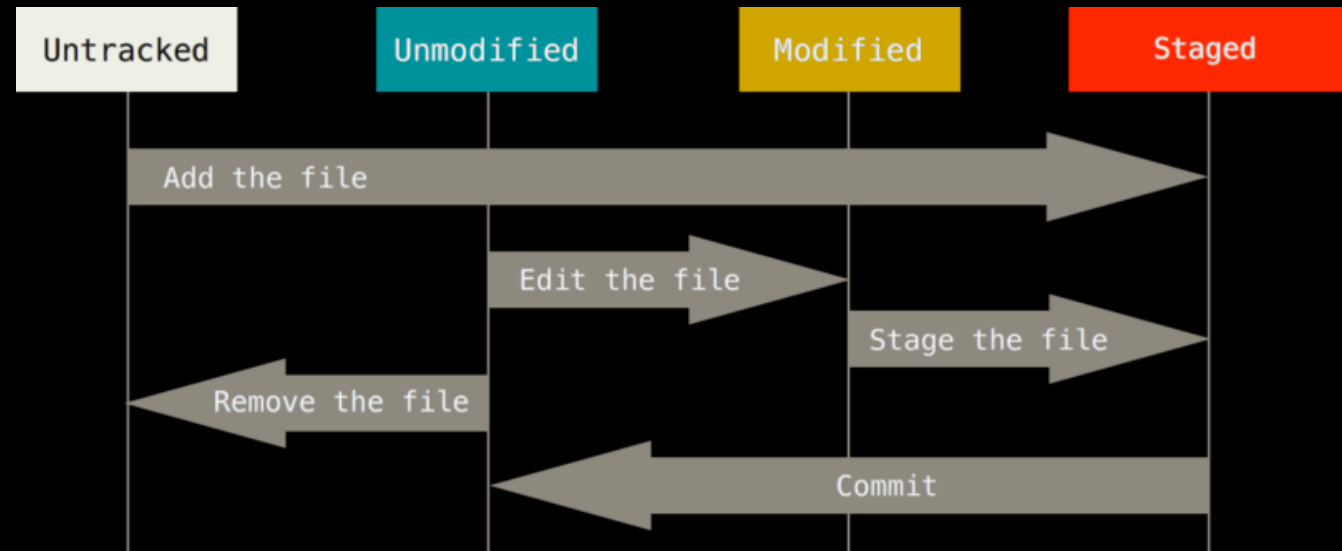
Stage the changes that are
to be part of the next commit



Commit the changes
present in the staging area
to the Git directory as a new
snapshot

Working with the files

- Files can be in two states, tracked or untracked
- Tracked files are files that were in the last snapshot
 - They can be unmodified, modified or staged
- Untracked files are files that Git is not aware of - therefore they are not in the scope of the version control
 - A special file called “.gitignore” can be used to inform git about files to leave untracked



Useful Git commands

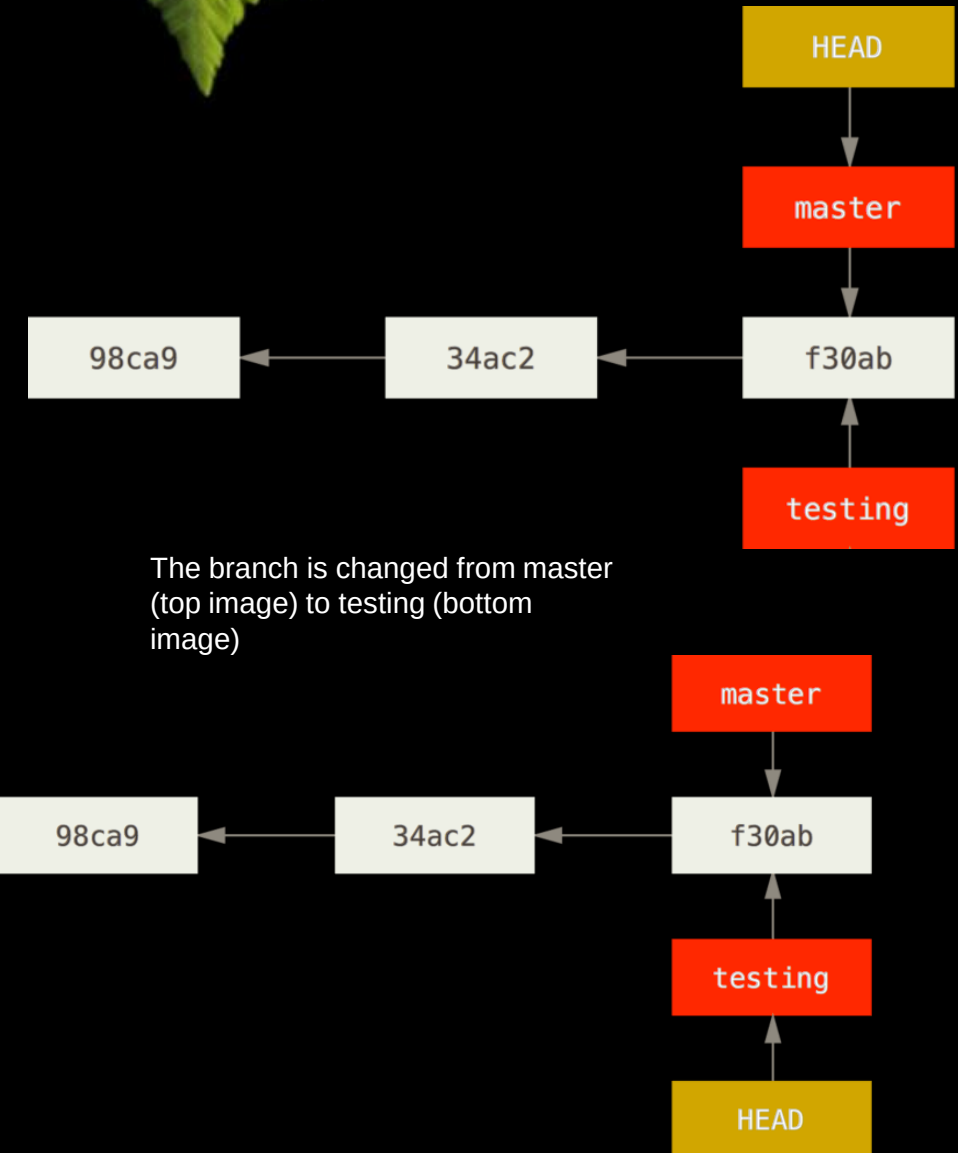
- Initializing a new repository: `git init`
 - Cloning an existing repository: `git clone <url>`
 - Checking the status of your files: `git status`
 - Begin to track a new file: `git add`
 - Stage a modified file: `git add`
 - View the changes that are not yet staged: `git diff`
 - Commit changes: `git commit`
 - Remove a file from Git: `git rm`
 - Move / rename files: `git mv`
- As it can be seen, `git add` is a multipurpose command

Committing

- When a commit is made, Git stores a “commit object” that includes a pointer to the snapshot of the staged content of that commit
 - Committing essentially means to “save” the changes that are made to the files
 - Only the staged changes are added to the commit -> if you have created or modified files after staging, they won't be a part of the commit unless you stage them again
- When committing, it is possible to leave a “commit message” that briefly states what changes have been made in this commit
 - Eg. “fixed a typo in the front page”
- Every commit has an unique “id”
 - It is possible to go back in the version history by checking out a certain commit

Branching

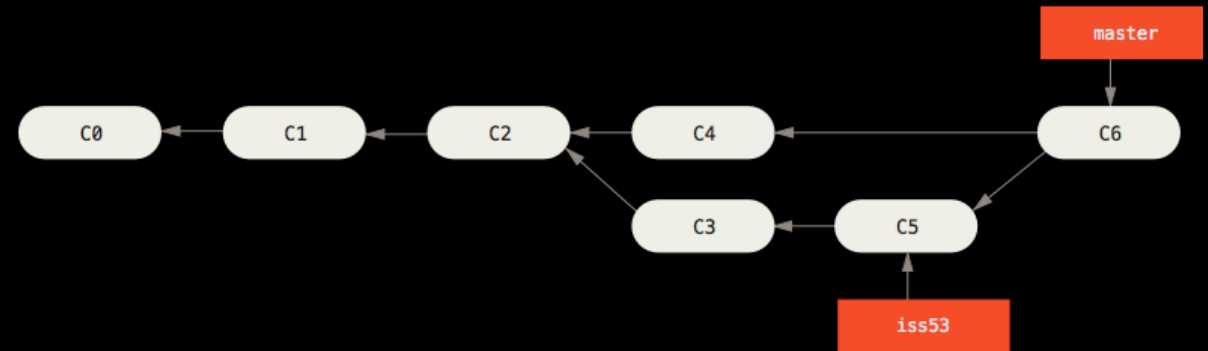
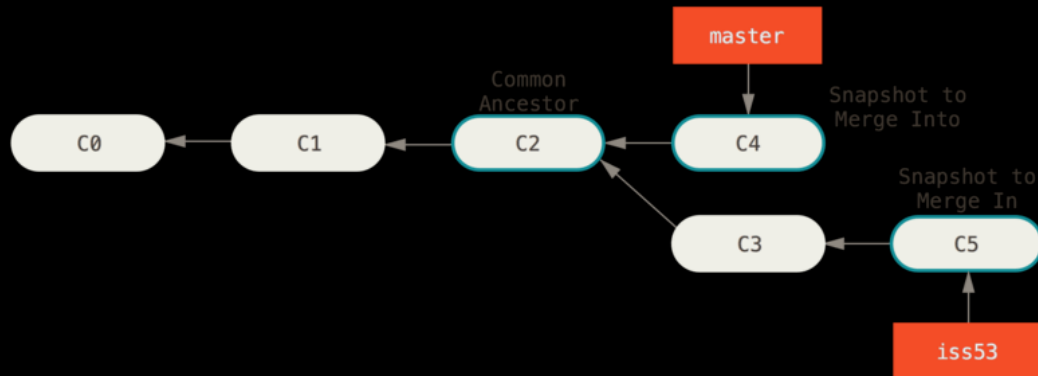
- Branching means the act of diverging from the main “line” of development in the version control system
 - Allows to change and record changes safely without affecting the main version history
 - One of Git’s “killer” features - its way of branching outperforms most other VC systems
- A branch in Git is a movable pointer to a certain commit
- Git has a special HEAD pointer to keep track of the current branch
- A new branch can be created with the command “git branch <branchname>”
 - This creates a branch with a pointer to the current commit
- A default branch (usually called master) is created when a new repository is created
- To switch to a certain branch, command “git checkout <branchname>” can be used
 - This moves the HEAD pointer to <branchname>



Merging

- Merging means to join two or more development histories together
- One common usage of merging is to join a certain single branch into the main branch
- Basic merging is done with the “git merge <branch>” command - the command should be run in the branch where the changes are to be merged
- For example, to merge branch A into main branch, the command is “git merge A” and it is run while on the main branch
- The merging process varies based on the conditions of the development histories to be joined
 - If the main branch hasn't been updated with commits since branch A was originally created from it, the merge can be “fast-forwarded” and it doesn't require any special operations
 - If fast-forwarding isn't possible, Git does some background work and tries to join the histories without the need for manual resolving - an additional merge commit is made in this case
 - Sometimes the histories cannot be joined together without resolving “merge conflicts” - they occur when same part of the file has been changed in both of the branches (in this case in branches A and main)
 - Merge conflicts need to be resolved by the user - they are solved by deciding what the conflicted part of the file shall include after the merge
 - After the conflict is resolved, the merge commit can be performed

Merging



Benefits of branching

- Each developer can develop using their own separate branches and finally merge the changes into the main branch
 - This keeps the main branch “clean” - one common principle is that only working software is allowed to be merged into it
- Even in the case of one developer, branching is beneficial
- It is up to the judgement of the developers when a new branch should be created
 - Minor hotfixes might be more efficient to make directly to the main branch
- Since branching and merging is very efficient in Git, it is encouraged to use branches as much as possible

Git workflows

- Workflows are a way of using Git according to certain guidelines
 - It keeps the version control well-structured and consistent
 - Requires the participation of every developer
- To select the workflow to be used, the following things should be taken into consideration:
 - Amount and experience of developers
 - Ability to undo mistakes and errors
- This course presents two workflows, which are based on the concept of branching:
 - Feature Branch workflow
 - Gitflow workflow

Feature branch workflow

- Core idea: all feature development is done the dedicated branches instead of the main branch
 - Multiple developers can work on a particular feature without affecting the main branch
 - Main branch should (at least in theory) always contain a working version of the software -> this is important when designing CI / CD pipelines (pipelines are a later topic in this course)
- Special requests can be used to review the features before merging them
 - GitHub has pull requests, GitLab merge requests etc...
 - Developers can comment on each other's work
- Feature branches are created from the latest state of the main branch
 - Branches can be named in a structured way, for example using the ID's from a project tracking software (JIRA, Trello, etc.)

Gitflow workflow

- Extends the feature branch workflow - the idea is to provide a workflow that helps to implement DevOps practices like CI / CD (discussed later in this course)
- Defines a strict branching model
 - Feature branches
 - Branches for preparing, maintaining and recording releases
- Main branch is used for storing official releases
- A develop branch is used to integrate features
- Feature branches are merged to develop -> they don't interact with the main branch
- When develop branch has enough new features, a new release branch is created from it
 - New features are not added anymore - bugfixes, document generation etc. are allowed
- When the release branch is ready, it is merged to master
 - The usage of release branch allows for one development team to focus on polishing the release while another team(s) can focus on the features of the next release
- If an issue in the main branch is found, a hotfix branch is created from it
 - The fix to the issue is merged to both the main and the develop branch
- As it can be seen, Gitflow adds some extra rules -> is it most suitable for large software projects which utilize DevOps practices

 Introduction to DevOps

References

- [git handbook](#)
- [atlassian git workflow](#)

THANK YOU!

